Practical - 6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic search, filter by category/course, sorting, pagination, and handle loading/error states.

A. Theory & Analysis

1. How is JSON fetched, parsed, and rendered?

I fetched the data asynchronously using fetch('../data/events.json'). When the server returns the response, response.json() parses the JSON string into an array of objects. The script loops through the items and generates Bootstrap event cards dynamically inside the DOM container.

2. Which array methods are used for search, filter, sort, and pagination?

JavaScript array methods are used: .filter() searches titles and filters by category; .sort() sorts events by date or title; .slice((page - 1) * perPage, page * perPage) extracts the items for the current page; and .map() / .forEach() renders the card elements.

3. How are loading and error states handled?

A loading spinner is shown while the fetch request is pending. If the fetch fails or encounters a network issue, an error alert informs the user. If a search query produces zero results, a friendly 'No events found matching your criteria' card is displayed.

4. How is modularity maintained in JavaScript files?

The code in js/events.js is divided into dedicated modular functions: fetchEvents(), renderEvents(), applyFilterAndSearch(), setupPagination(), and updatePaginationControls(). This keeps data fetching, filtering, and rendering clean and separate.

B. Implementation Details

I connected the campus events page (pages/events.html) to an external JSON dataset (data/events.json) containing 15 campus events. I also prepared data/students.json and data/faqs.json with 15 records each as required by the syllabus. The script in js/events.js performs asynchronous data fetching, keyword searching, category filtering, ascending/descending date sorting, and interactive page switching.

C. Evidence of Work

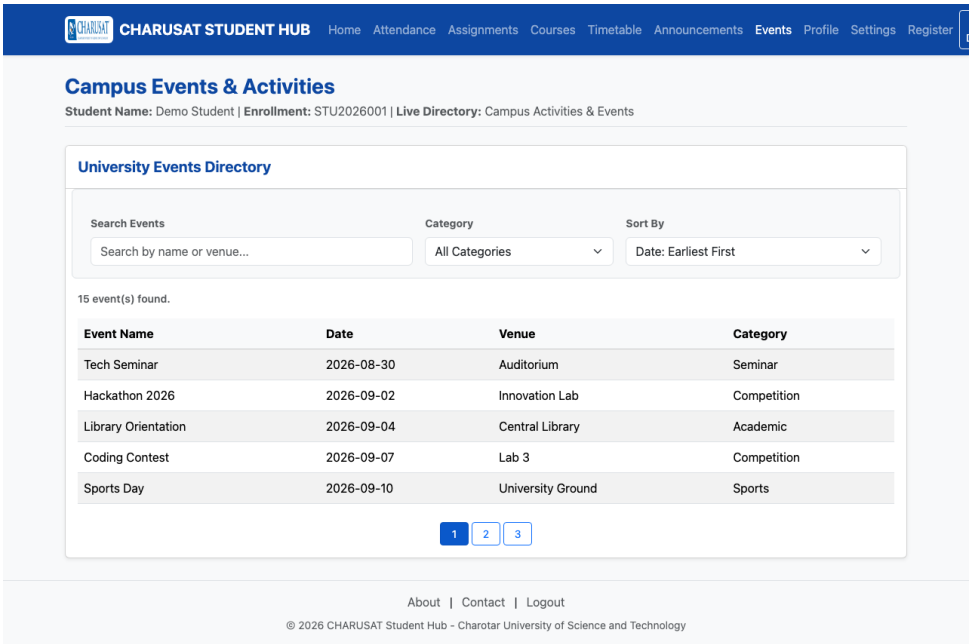


Figure: Campus Events Rendered Asynchronously from External JSON with Search and Filters

D. Repository Link

GitHub Repository: https://github.com/neel92654/WDF_A_ARP

E. Learning Outcomes

I learned how to use the Fetch API to load and parse external JSON datasets asynchronously, and how to apply array operations to build client-side search, filtering, sorting, and pagination.